LEZIONE 5

# TDD, BDD, ATDD e Specification by Example

Trasformare la spec in prove eseguibili. Durata indicativa: 30-45 minuti.

**Domain -> Requirements -> Spec -> Test -> Agent -> Verify**

Nelle lezioni precedenti abbiamo fatto tre cose diverse: abbiamo capito il dominio, trasformato quella conoscenza in requisiti verificabili e costruito una specification agent-ready. Oggi trasformiamo una parte della specification in feedback eseguibile. Il punto centrale non e "scrivere piu test", ma scegliere il loop di feedback corretto per il problema che stiamo risolvendo.

**Domanda guida: come passiamo da una regola concordata a prove eseguibili senza perdere il significato del dominio?**

## 1. Quattro concetti che spesso vengono confusi

TDD, BDD, ATDD e Specification by Example hanno aree di sovrapposizione, ma non sono sinonimi. Confonderli porta a errori pratici: usare Cucumber e chiamarlo BDD, scrivere test unitari prima del codice e pensare di aver fatto ATDD, oppure trasformare ogni esempio di business in un test end-to-end lento.

| Concetto | Categoria principale | Problema che affronta | Output tipico |
|---|---|---|---|
| TDD | Tecnica di programmazione/design | Come implementare un comportamento tramite feedback rapido | Test e codice evoluti con Red/Green/ |
| BDD | Processo di collaborazione e linguaggio | Come allineare business e tecnologia sul comportamento atteso | Esempi condivisi, spesso |
| ATDD | Approccio acceptance-driven | Come definire l accettazione prima dell implementazione | Acceptance criteria/test definiti prima |
| Specification by Example | Metodo collaborativo | Come rendere requisiti astratti concreti e verificabili | Esempi realistici, living documentatio |

Agile Alliance descrive TDD come un intreccio di coding, testing e design basato su un singolo test, il minimo codice per farlo passare e il refactoring successivo. Cucumber descrive BDD come un processo di collaborazione che passa per Discovery, Formulation e Automation. ATDD, invece, porta prospettive diverse a definire criteri e prove di accettazione prima di implementare. Specification by Example usa esempi realistici per colmare il divario tra stakeholder e team tecnico.

## 2. TDD: il loop interno di programmazione

Il modello mentale piu utile e piccolo e ripetibile:

```
RED
  Scrivi un singolo test che esprime un comportamento non ancora presente.

GREEN
  Scrivi il minimo codice ragionevole per far passare quel test.
```

```
REFACTOR
  Migliora il design mantenendo tutto verde.

REPEAT
```

La parola importante e "singolo". TDD non significa preparare trenta test, eseguirli tutti rossi e poi implementare per ore. Il feedback utile nasce da cicli piccoli. Ogni ciclo aggiunge una piccola capacita osservabile e costringe il design a emergere sotto pressione di esempi concreti.

TDD risolve soprattutto un problema di feedback sull implementazione e sul design. Non e, da solo, un metodo per scoprire cosa il prodotto debba fare. Se la business rule e ancora UNKNOWN, scrivere un test non risolve l incertezza: la congela.

> Regola pratica: prima autorizza il comportamento, poi automatizzalo.

### Un esempio framework-independent

Dalla nostra spec: un Ticket non puo essere chiuso se esiste un Intervention in stato PIANIFICATO. La regola e gia stata decisa. Ora possiamo guidare l implementazione.

```
# primo ciclo: il test esprime il comportamento
Given: Ticket con un Intervention PIANIFICATO
When:  viene richiesta la chiusura
Then:  la chiusura viene rifiutata
```

Il test non dovrebbe chiedere se un certo metodo privato e stato chiamato tre volte. Dovrebbe osservare il comportamento attraverso una seam stabile: per esempio un application service, un port di dominio o un endpoint, a seconda di cio che abbiamo concordato nella spec.

## 3. BDD: molto piu di Given/When/Then

Cucumber e esplicito: BDD e un modo di lavorare che riduce il divario tra business e persone tecniche attraverso collaborazione, piccoli incrementi e documentazione verificata contro il comportamento del sistema. Il ciclo quotidiano viene presentato come tre pratiche: Discovery, Formulation e Automation.

```
DISCOVERY
  Parliamo di esempi concreti per capire cosa serve.
      |
      v
FORMULATION
  Documentiamo gli esempi in forma precisa e condivisa.
      |
      v
AUTOMATION
  Colleghiamo gli esempi al sistema e li eseguiamo.
```

Questo collega direttamente le nostre lezioni: EventStorming e Example Mapping appartengono soprattutto alla Discovery; i requisiti e le specifiche preparano la Formulation; i test di accettazione portano alcuni esempi nella Automation.

Un errore comune e: "usiamo Gherkin, quindi facciamo BDD". Non basta. Gherkin e una grammatica strutturata; Cucumber e un tool che esegue specifiche; BDD e il processo socio-tecnico che costruisce comprensione condivisa e la mantiene verificabile.

# 4. ATDD: accettazione prima dell implementazione

ATDD sposta in avanti la domanda "come sapremo che questa feature e accettabile?". Le persone con prospettive diverse collaborano prima dell implementazione per definire criteri ed esempi di accettazione. L Agile Practice Guide lo descrive esplicitamente come collaborazione tra prospettive customer, development e testing prima di costruire la funzionalita.

La differenza rispetto a TDD e soprattutto la scala e l autorita della prova. Un acceptance example esprime un risultato riconoscibile dal punto di vista dell utente o dello stakeholder; un test TDD interno puo guidare una piccola decisione di design.

| Domanda | ATDD / acceptance loop | TDD / developer loop |
|---|---|---|
| Chi definisce la prova? | Team cross-functional con fonte di dominio | Sviluppatore/agente durante l implementazione |
| Livello | Capability/comportamento osservabile | Seam tecnica o domain seam |
| Quando nasce? | Prima dell implementazione | Durante l implementazione, un ciclo alla volta |
| Scopo principale | Costruire la cosa giusta | Costruire bene la cosa concordata |

# 5. Specification by Example: esempi come ponte

Specification by Example, resa popolare da Gojko Adzic, e un metodo collaborativo per specificare requisiti e test tramite esempi realistici. Il suo valore non e soltanto automatizzare: e usare casi concreti per scoprire interpretazioni incompatibili, raffinare il linguaggio e creare documentazione viva.

Nella nostra pipeline possiamo pensare cosi:

```
RULE
  Un Ticket non puo essere chiuso con Intervention aperti.

EXAMPLES
  [COMPLETATO]                -> chiudibile
  [COMPLETATO, ANNULLATO]     -> chiudibile
  [COMPLETATO, PIANIFICATO] -> rifiutato

QUESTION
  [] -> un Ticket senza Intervention e chiudibile?
```

Rule + Examples e piu robusto di uno dei due da solo. La Rule generalizza. Gli Examples eliminano interpretazioni plausibili ma sbagliate. La Question conserva l incertezza senza costringere l agente a inventare.

## 6. Example Mapping e il ponte verso acceptance test

Example Mapping, introdotto nella Lezione 2, usa Story, Rules, Examples e Questions per chiarire l acceptance prima dello sviluppo. Cucumber lo presenta esplicitamente come una tecnica che rende breve e produttiva la conversazione e usa esempi concreti come base per acceptance tests.

```
STORY: Chiudere un Ticket

RULE: Nessun Intervention aperto

EXAMPLE A: COMPLETATO -> chiudibile
EXAMPLE B: PIANIFICATO -> rifiutato

QUESTION: ANNULLATO e uno stato terminale?
```

Quando la domanda su ANNULLATO viene risolta, l esempio puo diventare un criterio di accettazione e, se vale la pena automatizzarlo, una prova eseguibile.

## 7. Given/When/Then: notazione, non metodologia

Given/When/Then e una forma utile per strutturare un esempio. Il suo valore e separare contesto iniziale, evento e risultato osservabile. Non e obbligatorio usare Cucumber, e non e necessario scrivere ogni test in Gherkin.

```
Scenario: closure rejected while an intervention is planned

Given a Ticket with one Intervention in PIANIFICATO
When the Manager requests Ticket closure
Then the Ticket remains OPEN
And the closure request is rejected
```

La documentazione Cucumber raccomanda di descrivere il comportamento, non la procedura tecnica. "When Bob logs in" e piu resistente del dettaglio di campi, bottoni e URL quando quei dettagli non fanno parte della regola di business. Questo principio e prezioso anche per gli agenti: test accoppiati a dettagli interni amplificano il costo del refactor.

## 8. Outer loop + inner loop: il modello piu utile

Possiamo combinare ATDD/BDD e TDD con due loop concentrici. L outer loop parte da un esempio di accettazione che fallisce. Dentro, l implementazione procede con piccoli cicli TDD. Quando la capability e completa, anche l outer test diventa verde.

```
OUTER LOOP
  Acceptance example RED
        |
        +--> INNER LOOP
               unit/domain test RED
               minimal code GREEN
               REFACTOR
               repeat...
        |
        v
  Acceptance example GREEN
```

Questo modello risponde a due domande diverse: "stiamo costruendo la cosa giusta?" e "la stiamo costruendo con un design sostenibile?". Nessuno dei due loop elimina il bisogno dell altro.

## 9. Test seams: dove osserviamo il comportamento

Nella Lezione 4 abbiamo introdotto le test seams. Ora diventano operative. Una seam e un punto stabile attraverso cui possiamo esercitare e osservare una capability.

| Seam | Quando e utile | Rischio se abusata |
|---|---|---|
| Domain object / pure function | Regole pure e veloci | Troppo interna se la feature e soprattutto inte |
| Application service / use case | Ottimo compromesso per capability applicative | Puo richiedere fake di boundary esterni |
| HTTP/API | Contratto pubblico o integrazione importante | Piu lento e setup piu pesante |
| Browser/UI | Comportamento utente realmente end-to-end | Feedback lento, fragile se usato per ogni ciclo |

Il principio non e "testa sempre al livello piu alto possibile" in senso assoluto. E: scegli la seam piu alta e stabile che fornisce feedback rapido e significativo per quel comportamento. Poi aggiungi test piu larghi solo dove il rischio di integrazione lo giustifica.

## 10. Un comportamento, tre linguaggi

Lo stesso comportamento puo essere guidato in PHP, Python o TypeScript. La sintassi cambia; la regola e la fonte dell expected no.

```php
// PHP / Pest
it('rejects closure with an open intervention', function () {
    $ticket = Ticket::withIntervention('PIANIFICATO');

    expect(fn () => $ticket->close())
        ->toThrow(CannotCloseTicket::class);
});
```

```python
# Python / pytest
def test_rejects_closure_with_open_intervention():
    ticket = Ticket.with_intervention('PIANIFICATO')

    with pytest.raises(CannotCloseTicket):
        ticket.close()
```

```typescript
// TypeScript / Vitest
it('rejects closure with an open intervention', () => {
  const ticket = Ticket.withIntervention('PIANIFICATO')

  expect(() => ticket.close()).toThrow(CannotCloseTicket)
})
```

La cosa da verificare non e quale libreria preferiamo. La domanda e: questo expected deriva dalla spec? Il test osserva una capability o un dettaglio? Se rinomino una funzione interna senza cambiare comportamento, il test dovrebbe restare valido?

## 11. Test che aiutano davvero un agente

La documentazione corrente della skill /tdd di Matt Pocock insiste su alcuni segnali di qualita molto coerenti con il percorso del corso: test name orientato alla capability, expected values tracciabili alla spec, seam concordate prima del test e mock principalmente ai confini esterni.

- Capability names: preferisci "user can checkout with valid cart" a "checkout calls paymentService.process".
- Expected indipendente: non ricalcolare nel test lo stesso algoritmo del codice, altrimenti test e produzione possono sbagliare insieme.
- Red reale: il nuovo test deve fallire per la ragione attesa prima del codice.
- Un ciclo alla volta: evita batch di test rossi seguiti da un grande batch di implementazione.
- Mock ai boundary: payment provider, clock, filesystem o rete sono candidati naturali; i moduli interni non vanno automaticamente mockati.

## 12. Quando NON usare TDD o acceptance automation come martello

Non ogni cambiamento trae lo stesso vantaggio dallo stesso loop. Il criterio e il rapporto tra valore del feedback e costo della prova.

| Situazione | Scelta piu ragionevole |
|---|---|
| Cambio di testo o stile puramente cosmetico | Verifica visiva o test mirato; TDD potrebbe essere artificiale |
| Wiring/config senza fonte indipendente di verita | Evita test tautologici che ripetono l implementazione |
| Regola di dominio ancora UNKNOWN | Torna a Discovery/Requirements; il test non decide la policy |
| Browser test lento per ogni micro-ciclo | Usa una seam piu rapida per l inner loop; conserva pochi E2E ad alto valore |
| Dipendenza esterna | Isola il boundary; piu avanti useremo contract testing |

Un test e una forma di evidenza. Se l evidenza non e indipendente dall implementazione, rischia di diventare una tautologia.

## 13. Anti-pattern: test verdi, prodotto sbagliato

Torniamo al principio della Lezione 1: test verdi non implicano prodotto corretto. Considera una rule sbagliata inserita in spec: "un Ticket puo essere chiuso anche con un Intervention PIANIFICATO". Possiamo scrivere una suite perfetta che verifica esattamente questa policy. Tutto verde. Prodotto sbagliato.




Ecco perche la pipeline intera e importante. Test e agenti non sostituiscono Domain Discovery e Requirements Engineering; li rendono verificabili.

# 14. Workflow Matt Pocock: il TDD come engine di implement

Nel repository corrente delle engineering skills di Matt Pocock, la catena principale resta grill-with-docs -> to-spec -> to-tickets -> implement -> code-review. La documentazione di /tdd lo presenta come il motore del build step: /to-spec concorda le test seams, /implement guida /tdd per ticket e /code-review verifica dopo il lavoro.




Questa collocazione e interessante per un motivo teorico: la seam non dovrebbe essere inventata durante il test. E una decisione di design/verifica che idealmente nasce prima, nella spec, e deve sopravvivere alla decomposizione in ticket.

Il repository ha anche un implement-spec orientato a lavoro su piu ticket con subagent e worktree: ogni implementer lavora su un ticket in un worktree e dovrebbe guidare TDD. Questo anticipa il blocco successivo, in cui parleremo di orchestrazione agentica e isolamento del lavoro.

# 15. Classificazione: metodologia, tecnica, notazione o tool?

| Elemento | Categoria |
|---|---|
| TDD | Tecnica / disciplina di programmazione e design |
| BDD | Processo / approccio collaborativo di sviluppo |
| ATDD | Approccio acceptance-first collaborativo |
| Specification by Example | Metodo collaborativo di requirements e validation |
| Example Mapping | Tecnica di discovery/requirements |
| Given/When/Then | Notazione per strutturare esempi |
| Gherkin | Linguaggio/grammatica strutturata |
| Cucumber | Tool che esegue specifiche Gherkin |
| Pest / PHPUnit / pytest / Vitest | Test framework |
| Matt Pocock /tdd | Skill agentica che codifica una disciplina TDD |

## 16. Esercizio pratico - circa 25 minuti

Riprendi la specification agent-ready della Lezione 4. Se non hai svolto l esercizio, usa la capability "Close Ticket". L obiettivo e creare un outer example e almeno un piccolo inner loop TDD senza aggiungere business logic nuova.

- Minuti 0-5: scegli una Rule gia DECIDED e scrivi due Examples concreti: uno happy path e uno failure path.
- Minuti 5-9: trasforma il failure path in Given/When/Then technology-neutral. Se compare una domanda, registrala come UNKNOWN e non proseguire su quel ramo.
- Minuti 9-12: scegli la verification seam. Scrivi una riga: "Verifico questa capability attraverso X perche...".
- Minuti 12-18: scrivi un singolo test nel linguaggio che preferisci. Eseguilo e assicurati che sia RED per la ragione corretta.
- Minuti 18-22: scrivi il minimo codice necessario per arrivare a GREEN.
- Minuti 22-25: refactor e controlla la traceability Rule -> Example -> Test -> Code. Qualsiasi nuova policy emersa dal codice deve tornare a Requirements/Spec.

Criterio di successo: un altro sviluppatore deve poter risalire dal test alla Rule senza dover indovinare perche quell expected esiste.

## 17. Takeaway della Lezione 5




**Frase da ricordare: un test verde dimostra che il sistema soddisfa quella prova; non dimostra, da solo, che la prova rappresenti il bisogno corretto.**

Con questa lezione abbiamo completato il primo blocco operativo del corso: Lezioni 2-5. Il passaggio successivo e architetturale: dobbiamo capire come creare confini e seam che rendano queste prove stabili e l implementazione sostituibile.

# HAND-OFF COMPLETO - fine Blocco 1 (Lezioni 2-5)

Usa questa sezione come contesto iniziale della prossima chat.

## Stato del percorso

Lezione 1 gia completata prima di questo blocco: "Dal far scrivere codice all AI al fare Software Engineering con l AI". Ha introdotto problem space vs solution space, Domain -> Requirements -> Spec -> Test -> Agent -> Verify, invarianti, KNOWN/ASSUMED/UNKNOWN/DECIDED, test verdi != prodotto corretto, AI come implementatore e non autorita di dominio, e il workflow Matt Pocock come filo conduttore.

Blocco appena completato: Lezioni 2, 3, 4 e 5.

## Lezione 2 - Domain Discovery: capire il problema prima di specificarlo

- DDD come disciplina centrata su linguaggio, modello e confini; Ubiquitous Language e Bounded Context.
- EventStorming, Domain Storytelling ed Example Mapping classificati come tecniche/metodi, non tool o skill AI.
- Discovery orientato a eventi, storie, regole, esempi, domande e confini.
- CONTEXT.md, glossary, open questions e ADR come artefatti durevoli.
- grill-with-docs / domain-modeling come pratiche agentiche che aiutano a rendere esplicite ambiguita e decisioni.

## Lezione 3 - Requirements Engineering: dal dominio a requisiti verificabili

- Separazione Business Goal / Stakeholder Requirement / Functional Requirement / NFR / Constraint / Invariant.
- EARS come tecnica di controlled natural language, non metodologia completa.
- User Story, EARS e Given/When/Then come rappresentazioni complementari.
- Traceability: Evidence -> Rule -> Requirement -> Example/Test -> Code.
- Requirement laundering: una ASSUMPTION non diventa FACT soltanto perche e stata riscritta in forma formale.

## Lezione 4 - Spec-Driven Development: dai requisiti a una spec agent-ready

- Distinzione Requirement / Specification / Design / Plan / Ticket.
- Specification come compressione intenzionale del contesto e interfaccia tra intent umano e agent implementation.
- Spec agent-ready: scope, glossary, invariants, examples, non-goals, UNKNOWN, test seams.
- Spec Kit, OpenSpec, Kiro, Tessl, Superpowers e Matt Pocock skills classificati per categoria e filosofia.

- to-spec cattura decisioni gia convergenti; to-tickets crea tracer-bullet vertical slices.
- Spec drift, mega-spec, design leakage, pseudo-precision e template worship come anti-pattern.

## Lezione 5 - TDD, BDD, ATDD e Specification by Example

- TDD = inner programming/design feedback loop: Red -> Green -> Refactor.
- BDD = collaborazione sul comportamento: Discovery -> Formulation -> Automation; non coincide con Gherkin/Cucumber.
- ATDD = criteri/prove di accettazione definiti prima dell implementazione da prospettive diverse.
- Specification by Example = metodo collaborativo basato su esempi realistici e living documentation.
- Outer acceptance loop + inner TDD loop.
- Test seam come punto stabile di verifica; expected values devono essere tracciabili alla spec.
- Nel workflow Matt Pocock /tdd e il motore dentro /implement e lavora a seam pre-concordate.

## Principi trasversali da preservare

- L AI non decide le business policy mancanti: UNKNOWN resta UNKNOWN.
- La formalita di un documento non aumenta l autorita epistemica di una decisione.
- Rule + Examples e preferibile a sola Rule o soli Examples.
- Spec e test devono descrivere comportamento e vincoli autorizzati, non accoppiarsi inutilmente ai dettagli interni.
- Il processo va proporzionato a rischio e decision density: non ogni modifica richiede spec completa o TDD formale.
- La traceability deve poter collegare dominio, requirement, spec, example/test e codice.

## Esercizi svolti o pendenti

Gli esercizi delle lezioni sono volutamente incrementali e possono usare la stessa capability, ad esempio Close Ticket. Se non completati, non bloccano il corso, ma il laboratorio ideale produce questi artefatti:

- timeline/eventi + glossary + KNOWN/ASSUMED/UNKNOWN/DECIDED;
- 3-5 requirements con almeno un EARS e un NFR/UNKNOWN;
- una piccola spec agent-ready con scope, non-goals, invariants, examples e seam;
- un acceptance example e un ciclo TDD completo Rule -> Example -> Test -> Code.

## Punto esatto da cui ripartire

Lezione 6 - Clean / Hexagonal Architecture: creare confini e test seams stabili.

La lezione deve partire dal problema emerso oggi: test e agenti funzionano meglio quando il sistema espone seam stabili. La prossima tappa e quindi capire entities/use

cases/ports/adapters, dependency inversion, boundaries, deep modules e come evitare che framework e I/O contaminino il core. Usare esempi equivalenti in PHP/Laravel, Python/FastAPI o Django e TypeScript/Node senza trasformare il corso in un tutorial di framework.

### Obiettivi proposti per il prossimo blocco (Lezioni 6-10)

- Lezione 6: Clean/Hexagonal Architecture, ports/adapters, dependency inversion e test seams.
- Lezione 7: Contract testing e property-based testing: verificare boundary e invarianti oltre gli esempi manuali.
- Lezione 8: Context engineering: AGENTS.md, CONTEXT.md, ADR, context hygiene, durable context e handoff.
- Lezione 9: Skills, subagents, worktree, Superpowers e Ralph loops: orchestrare agenti senza perdere controllo.
- Lezione 10: Verification ed Evaluation-Driven Development: eval, rubriche, regression gates e chiusura del ciclo Agent -> Verify.

Dopo il prossimo blocco si potra dedicare un ulteriore blocco a confronti e laboratori approfonditi su Spec Kit, OpenSpec, Kiro, Tessl, Superpowers e workflow multi-agent, mantenendo sempre la teoria di software engineering come riferimento.

## Fonti autorevoli e letture


Agile Alliance - TDD
https://agilealliance.org/glossary/tdd/

Cucumber - Behaviour-Driven Development
https://cucumber.io/docs/bdd/

Cucumber - Example Mapping
https://cucumber.io/docs/bdd/example-mapping/

Cucumber - Writing better Gherkin
https://cucumber.io/docs/bdd/better-gherkin/

Agile Practice Guide / Agile Alliance - definizione ATDD
https://www.agilealliance.org/wp-content/uploads/2017/08/AgileExtension_V2-Member-Copy.pdf

Gojko Adzic - Specification by Example (Manning)
https://www.manning.com/books/specification-by-example

Martin Fowler - Mocks Aren't Stubs
https://martinfowler.com/articles/mocksArentStubs.html

Martin Fowler - The Practical Test Pyramid
https://martinfowler.com/articles/practical-test-pyramid.html

Matt Pocock - skills repository
https://github.com/mattpocock/skills

Matt Pocock - TDD docs
https://github.com/mattpocock/skills/blob/main/docs/engineering/tdd.md

Matt Pocock - to-spec docs
https://github.com/mattpocock/skills/blob/main/docs/engineering/to-spec.md

Matt Pocock - implement docs
https://github.com/mattpocock/skills/blob/main/docs/engineering/implement.md


Fine Lezione 5 - fine del primo blocco di 4 lezioni in questa chat.